

A Deep Dive into Rainbow DQN

Chapter 6: Noisy Nets for Exploration

Harsh Verma

June 26, 2025

Abstract

Effective exploration remains a cornerstone of reinforcement learning. The traditional ϵ -greedy strategy, while simple, is fundamentally inefficient as its exploration is random and decoupled from the agent's knowledge. This chapter introduces Noisy Networks (Noisy Nets), proposed by Fortunato et al. (2017), a technique that replaces this naive approach with learned, state-dependent exploration. By adding parametric noise directly to the network's weights, the agent learns to control its own policy's stochasticity, leading to far more efficient and robust exploration strategies.

1 The Motivation: The Failures of ϵ -Greedy

The exploration-exploitation dilemma is central to RL. The standard solution, ϵ -greedy, involves taking a random action with probability ϵ and the greedy action otherwise. While foundational, it has severe limitations:

- **State-Agnostic Exploration:** The decision to explore is completely random and independent of the state. The agent is equally likely to take a random action in a critical, well-understood situation as it is in a novel one. A smart agent should explore more when it is uncertain.
- **Indirectness:** The agent doesn't learn *how* to explore. It simply learns a deterministic policy, and an external mechanism forces it to occasionally deviate.
- **Hyperparameter Sensitivity:** Performance is highly dependent on the ϵ annealing schedule (the rate at which ϵ decays). Finding an optimal schedule is non-trivial and environment-specific.

Noisy Nets address these issues by making exploration an intrinsic property of the agent's network.

2 The Noisy Net Solution: Parametric Noise in Weight Space

2.1 The Core Idea: From Action Space to Weight Space

The key insight of Noisy Nets is to shift the source of randomness. Instead of adding noise to the final actions, we add learnable noise directly to the `**weights` of the network's linear layers`**`.

The intuition is that the network can learn to control the magnitude of this internal noise. In states where it is uncertain, it can learn to "turn up" the noise, making its policy more stochastic and thus more exploratory. In states where it is confident, it can learn to "turn

down” the noise, leading to a deterministic, exploitative policy. Because the network’s output is a function of the input state, this exploration automatically becomes **state-dependent**.

2.2 Mathematical Formulation of a Noisy Linear Layer

A standard deterministic linear layer is defined by the affine transformation $y = wx + b$. A Noisy Linear Layer redefines the weight matrix w and bias vector b as random variables that are perturbed by learnable noise.

The weight and bias are parameterized by a mean component (μ) and a standard deviation component (σ):

$$w = \mu_w + \sigma_w \odot \epsilon_w \quad (1)$$

$$b = \mu_b + \sigma_b \odot \epsilon_b \quad (2)$$

where $\odot$ is element-wise multiplication. The full layer transformation is thus:

$$y = (\mu_w + \sigma_w \odot \epsilon_w)x + (\mu_b + \sigma_b \odot \epsilon_b) \quad (3)$$

- μ_w, μ_b : These are the learnable mean parameters, analogous to standard weights and biases. They form the deterministic core of the layer.
- σ_w, σ_b : These are the **learnable standard deviation** parameters. These are the most important new addition. The network learns the magnitude of the noise to apply via backpropagation. If the network learns a large σ for a particular weight, that weight will be very noisy.
- ϵ_w, ϵ_b : These are non-learnable **random noise variables**, typically sampled from a unit Gaussian distribution. They provide the source of randomness and are resampled at regular intervals (e.g., once per learning step).

This formulation is powerful because the network can learn to control its own level of exploration by adjusting the σ parameters through the standard gradient descent process.

2.3 Factorised Gaussian Noise

To improve computational efficiency, Noisy Nets use **Factorised Gaussian Noise**. Instead of sampling a full noise matrix ϵ_w , two smaller noise vectors, ϵ_i and ϵ_j , are sampled. The final noise matrix is then constructed from their outer product: $\epsilon_{i,j} = f(\epsilon_i) \cdot f(\epsilon_j)$, where $f(x) = \text{sgn}(x)\sqrt{|x|}$. This dramatically reduces the number of random numbers that need to be generated.

3 Implementation and Visualization

3.1 Architectural Diagram

Figure ?? illustrates the internal components of a Noisy Linear layer.

3.2 PyTorch Implementation

The implementation involves creating a custom ‘nn.Module’ to replace ‘nn.Linear’.

```
1 import torch
2 import torch.nn as nn
3 import math
4
5 class NoisyLinear(nn.Module):
```

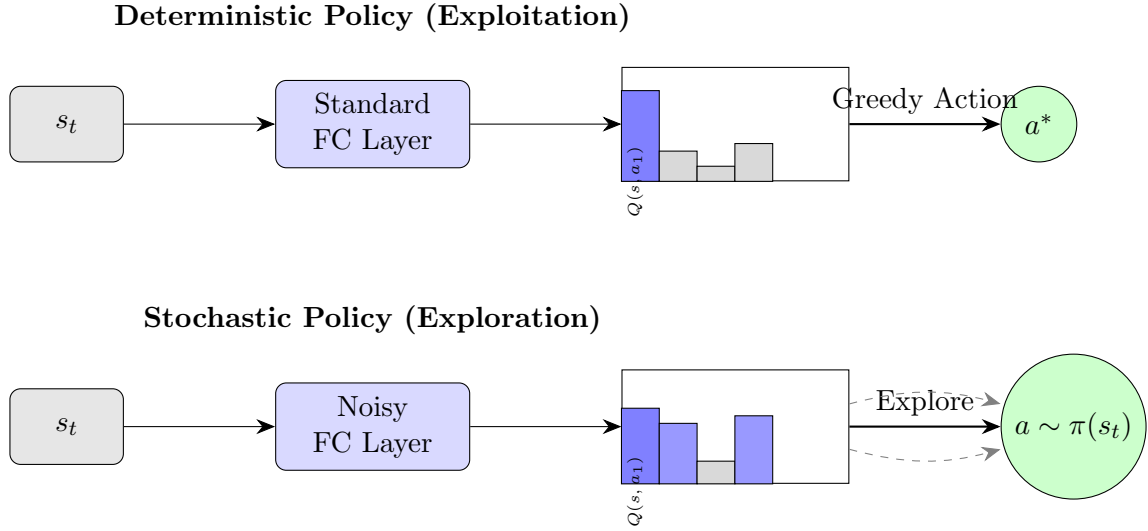


Figure 1: A conceptual diagram comparing a standard layer to a Noisy Net layer. The standard layer produces a deterministic policy by always choosing the action with the highest Q-value. The Noisy Net layer perturbs its weights, leading to a stochastic distribution of Q-values, which naturally induces exploration.

```

6  def __init__(self, in_features, out_features, std_init=0.5):
7      super(NoisyLinear, self).__init__()
8      self.in_features = in_features
9      self.out_features = out_features
10     self.std_init = std_init
11
12     # Learnable mean parameters
13     self.weight_mu = nn.Parameter(torch.empty(out_features, in_features))
14     self.bias_mu = nn.Parameter(torch.empty(out_features))
15
16     # Learnable standard deviation parameters
17     self.weight_sigma = nn.Parameter(torch.empty(out_features, in_features))
18 )
19     self.bias_sigma = nn.Parameter(torch.empty(out_features))
20
21     # Buffers for the noise (not learned, just stored)
22     self.register_buffer('weight_epsilon', torch.empty(out_features,
23 in_features))
24     self.register_buffer('bias_epsilon', torch.empty(out_features))
25
26     self.reset_parameters()
27     self.sample_noise()
28
29 def reset_parameters(self):
30     # Initialize parameters according to the paper
31     mu_range = 1 / math.sqrt(self.in_features)
32     self.weight_mu.data.uniform_(-mu_range, mu_range)
33     self.bias_mu.data.uniform_(-mu_range, mu_range)
34     self.weight_sigma.data.fill_(self.std_init / math.sqrt(self.in_features))
35 )
36     self.bias_sigma.data.fill_(self.std_init / math.sqrt(self.out_features))
37
38 def _scale_noise(self, size):
39     # Factorised Gaussian noise: f(x) = sgn(x) * sqrt(|x|)
40     x = torch.randn(size, device=self.weight_mu.device)
41     return x.sign() * x.abs().sqrt()

```

```

39
40 def sample_noise(self):
41     """Sample new noise using Factorised Gaussian Noise."""
42     epsilon_in = self._scale_noise(self.in_features)
43     epsilon_out = self._scale_noise(self.out_features)
44     # Outer product to create matrix from vectors
45     self.weight_epsilon.copy_(epsilon_out.ger(epsilon_in))
46     self.bias_epsilon.copy_(epsilon_out)
47
48 def forward(self, x):
49     if self.training:
50         # During training, use the noisy weights
51         weight = self.weight_mu + self.weight_sigma * self.weight_epsilon
52         bias = self.bias_mu + self.bias_sigma * self.bias_epsilon
53         return F.linear(x, weight, bias)
54     else:
55         # During evaluation, use only the mean parameters for a
56         deterministic policy
57         return F.linear(x, self.weight_mu, self.bias_mu)

```

Listing 1: PyTorch implementation of a NoisyLinear layer.

4 Discussion

- **A Complete Replacement:** Noisy Nets are not an add-on to ϵ -greedy; they are a complete replacement. When using Noisy Nets, the agent’s action selection is always greedy with respect to the network’s (now stochastic) output.
- **Synergy:** Noisy Nets are typically applied to the final fully-connected layers of the network (e.g., in the value and advantage streams of a Dueling architecture). This injects noise right before the final Q-values or distributions are computed, directly influencing the stochasticity of the final policy. This allows the learned uncertainty to directly drive exploration.

By integrating exploration directly into the learning process, Noisy Nets provide a more intelligent, efficient, and robust solution than traditional methods, forming the final cornerstone of the Rainbow agent.